API Specification Document

ReqAgent — AI Requirement Intake & Tracker

Author: Steven Saji Paul | Role: Business Systems Analyst

Version: 1.0 | Date: June 2026 | Status: Final

1. Overview

This document specifies the external API integrations used by ReqAgent. It covers the Groq LLM API, MongoDB Atlas connection, and Neon Postgres connection — including request/response formats, authentication, and error handling.

2. Groq LLM API

Property	Value
Endpoint	https://api.groq.com/openai/v1/chat/completions
Method	POST
Authentication	Bearer token (GROQ_API_KEY environment variable)
Model	llama-3.3-70b-versatile
Max Tokens	2000
Temperature	0.3 (low — for consistent, structured output)
Response Format	JSON (parsed from model text output)
Timeout	30 seconds
Retry Policy	1 retry on failure, then graceful error display

Request Payload Structure

```
{
"model": "llama-3.3-70b-versatile",
"messages": [
```

```
{
"role": "user",
"content": ""
}
],
"temperature": 0.3,
"max_tokens": 2000
}
```

Expected Response Structure

```
{
"user_stories": [
{
"story": "As a [user], I want [action] so that [benefit]",
"acceptance_criteria": "Given... When... Then..."
}
],
"moscow": "Must Have | Should Have | Could Have | Won't Have",
"priority": "High | Medium | Low",
"priority_justification": "One sentence explaining priority assignment.",
"conflicts": [
{
"conflicting_req_id": "REQ-xxxxxx or null",
"description": "Description of conflict or No conflicts detected."
}
]
}
```

3. MongoDB Atlas Connection

Property	Value
Connection Type	MongoDB SRV connection string
Library	pymongo 4.17.0
Authentication	Username + password in connection string
SSL	Enforced by default in Atlas
Database	requirement_tracker

Property	Value
Collection	raw_intake
Connection String Format	mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/?appName=<app>

Key Operations

Operation	Method	Description
Insert raw intake	collection.insert_one(document)	Saves requirement form data to MongoDB
Mark as processed	collection.update_one(filter, update)	Sets processed=True after AI completes
Get all titles	collection.find({}, {title:1, description:1})	Retrieves existing requirements for conflict check
Get by ID	collection.find_one({'_id': ObjectId(id)})	Retrieves single document for tracker view

4. Neon Postgres Connection

Property	Value
Connection Type	PostgreSQL connection string with SSL
Library	psycopg2-binary 2.9.x
Authentication	Username + password in connection string
SSL	sslmode=require enforced
Database	neondb
Connection String Format	postgresql://<user>:<pass>@<endpoint>/<db>?sslmode=require

Key Operations

Operation	SQL	Description
Insert requirement	INSERT INTO requirements (...)	Saves structured requirement after AI processing
Insert user stories	INSERT INTO user_stories (...)	Saves 3 stories per requirement
Update status	UPDATE requirements SET status = %s	Updates lifecycle status
Log status change	INSERT INTO status_audit (...)	Creates immutable audit trail entry
Get traceability	SELECT r.*, us.* FROM requirements r LEFT JOIN user_stories us ON r.id = us.requirement_id	Joins tables for complete view

5. Error Handling

Error Type	Cause	User Experience
Groq API Timeout	Response > 30 seconds	AI processing failed. Please try again.
JSON Parse Error	AI returns malformed JSON	Silent retry, then error if parse fails
MongoDB Auth Error	Invalid credentials	Database connection failed.
Postgres Connection Error	SSL/network issue	Could not save. Please retry.
Validation Error	Missing required fields	Red inline error message on form
Rate Limit Error	Groq API quota exceeded	Rate limit reached. Try in 60 seconds.